

Análisis de complejidad Big-O

Para analizar la complejidad del programa se consideran dos variables:

- **n**: número total de palabras procesadas.
- **u**: número de palabras distintas.

1. Normalización del texto

La función `TextNormalizer.normalize()` recorre el texto para convertirlo a minúsculas, eliminar acentos y separar las palabras.

Por lo tanto, su complejidad temporal es:

$O(n)$

ya que el número de operaciones aumenta proporcionalmente al tamaño del texto.

2. Tabla Hash

La función `add_or_increment()` se utiliza para contar la frecuencia de cada palabra. La búsqueda dentro de la tabla hash tiene una complejidad promedio de **$O(1)$** por palabra, suponiendo una distribución adecuada de las claves.

Como se procesan las **n** palabras:

$O(n)$ en promedio.

La función `get_unique_words_and_counts()` recorre las palabras almacenadas para obtener las palabras distintas, por lo que tiene una complejidad de:

$O(u)$

3. Merge Sort

Merge Sort divide la lista de palabras distintas en partes y posteriormente las combina de manera ordenada.

Su complejidad temporal es:

- Mejor caso: **$O(u \log u)$**
- Caso promedio: **$O(u \log u)$**
- Peor caso: **$O(u \log u)$**

Por lo tanto, Merge Sort mantiene una complejidad de **$O(u \log u)$** incluso en el peor caso.

4. Quick Sort

Quick Sort selecciona un elemento como pivote y divide las palabras en tres grupos: menores, iguales y mayores al pivote.

En el caso promedio, su complejidad es:

$O(u \log u)$

Sin embargo, si las particiones resultan muy desequilibradas, puede llegar al peor caso:

$O(u^2)$

5. Complejidad total

El programa primero procesa las **n** palabras y después ordena las **u** palabras distintas.

Por lo tanto, utilizando Merge Sort, la complejidad temporal total es:

$O(n + u \log u)$

Con Quick Sort, la complejidad promedio es:

$O(n + u \log u)$

y en su peor caso:

$O(n + u^2)$.

Análisis de los tiempos obtenidos

La ejecución del programa produjo los siguientes resultados:

Tamaño del texto	Palabras procesadas	Palabras distintas	Merge Sort (s)	Quick Sort (s)
~100 palabras	108	82	0.000257	0.000258
~1,000 palabras	1,080	82	0.000265	0.000267
~10,000 palabras	10,800	82	0.000268	0.000257

```
... ejecucion_comparacion()
...
Tamaño del texto | Palabras procesadas | Palabras distintas | Merge Sort (s) | Quick Sort (s)
-----
~100 palabras    | 108                 | 82                 | 0.000257       | 0.000258
~1,000 palabras  | 1080                | 82                 | 0.000265       | 0.000267
~10,000 palabras | 10800               | 82                 | 0.000268       | 0.000257
>>>
```

Se observa que el número de palabras procesadas aumenta de **108 a 10,800**, pero el número de palabras distintas permanece en **82**. Esto es importante porque los algoritmos de ordenamiento no ordenan las 10,800 palabras, sino únicamente las **82 palabras distintas** obtenidas después del conteo mediante la tabla hash.

Por esta razón, los tiempos de Merge Sort y Quick Sort permanecen muy cercanos entre las tres pruebas. Las pequeñas diferencias observadas en los tiempos de ejecución se deben a las condiciones de ejecución del programa y del sistema.

Conclusión

Los resultados experimentales son consistentes con el análisis teórico. **Merge Sort** presenta una complejidad de **$O(u \log u)$** en todos los casos, mientras que **Quick Sort** presenta **$O(u \log u)$** en promedio y puede alcanzar **$O(u^2)$** en el peor caso. Considerando todo el procesamiento del programa, la complejidad temporal promedio es **$O(n + u \log u)$** .

En las pruebas realizadas, el número de palabras distintas se mantuvo en **82**, por lo que los tiempos de ambos algoritmos de ordenamiento permanecieron muy similares, aunque el número total de palabras procesadas aumentó considerablemente.